

Advanced configuration



Deep dive in process and pipeline configuration

Before we start

Main folder for this section:

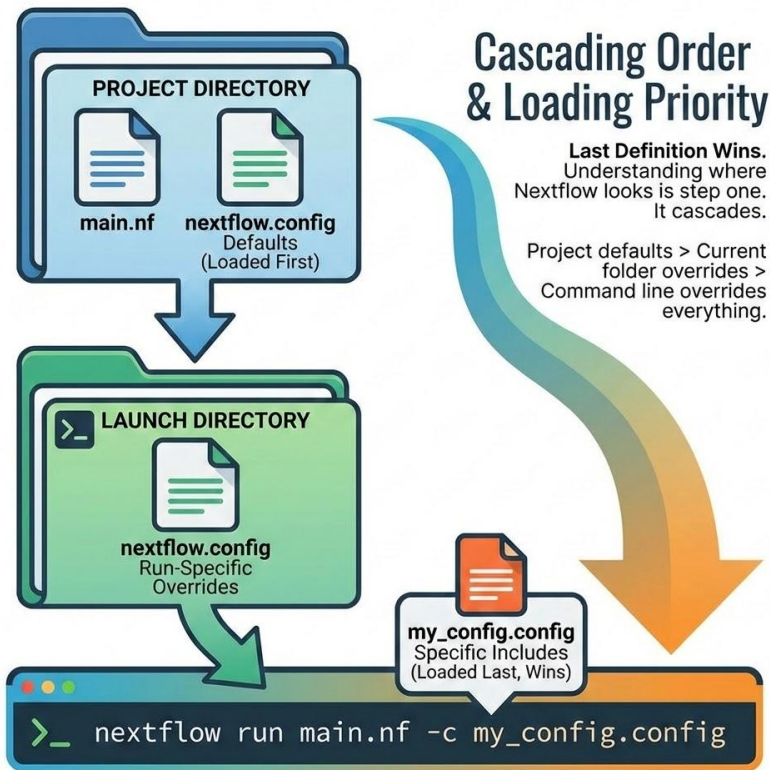
practicals/day3/1-advanced_config

Materials for this sections:

- **reference_documentation.md**: Reference documentation and code examples for the topics touched during this section
- **exercises.md**: Guide for the practical exercises

To run most of exercises you'll need a small interactive session with 4 CPUs and 12G memory. Specific instructions are given in the exercises document

Where does configuration live?



Where does configuration live?

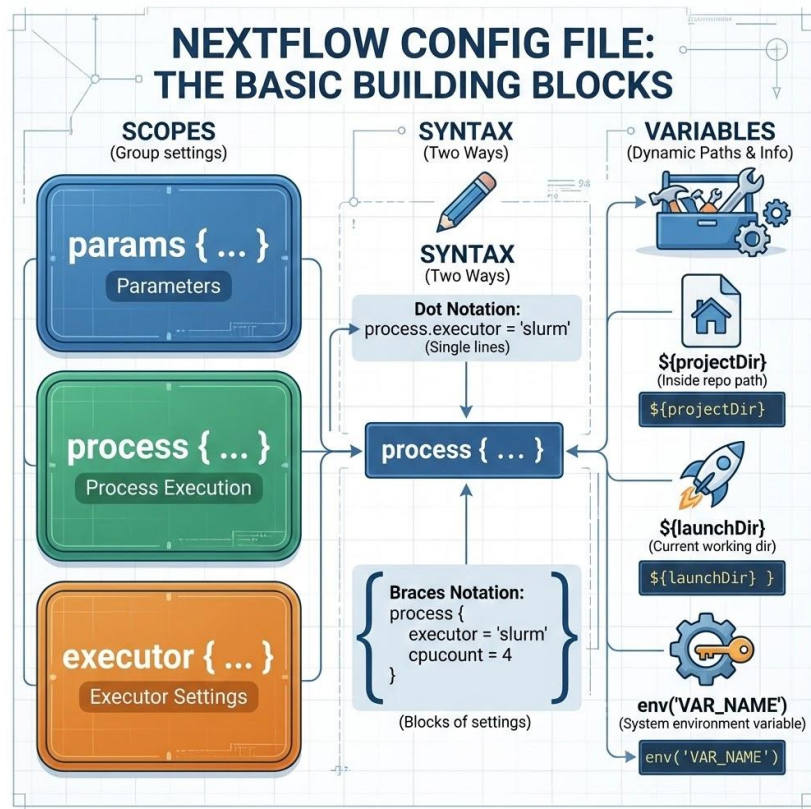
1. **Project Directory:** `nextflow.config` (next to `main.nf`). Always loaded.
2. **Launch Directory:** `nextflow.config` (where you run the command). Use this for run-specific overrides.
3. **Additional specific files:** included from CLI using `-c my_config.config` or from `nextflow.config` using `includeConfig 'file'`.

Priority matters!

Nextflow loads these in the order above.

Conflicting settings and parameters are overwritten by higher priority source

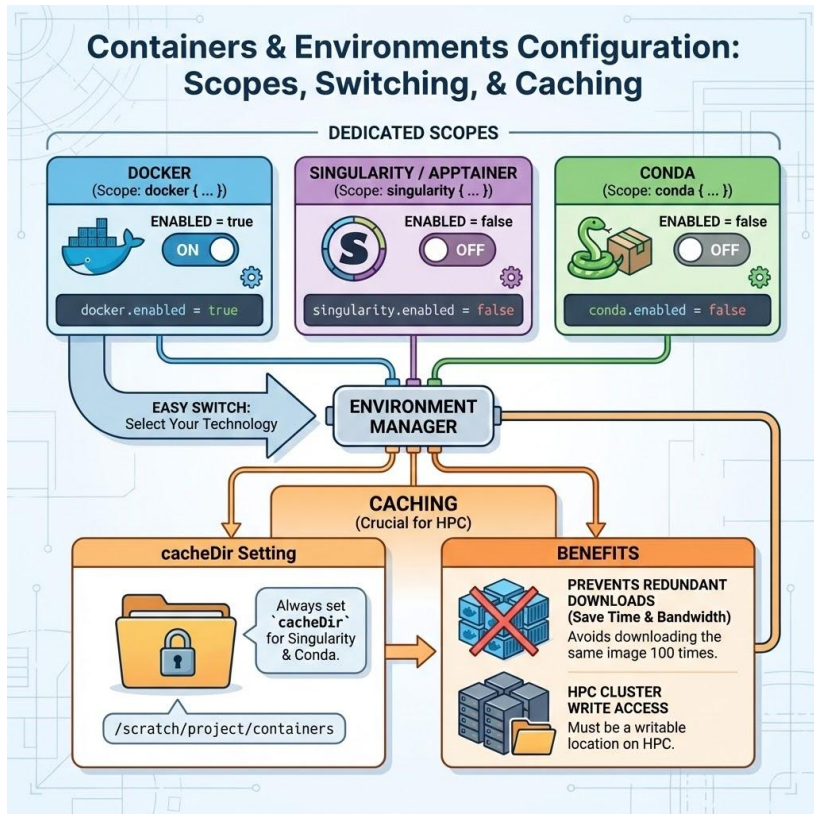
Scopes and Syntax



The Basic Building Blocks

- **Scopes:** Group settings by function (e.g., `params`, `process`, `executor`).
- **Syntax:**
 - **Dot notation:** `process.executor = 'slurm'` (Good for single lines).
 - **Braces notation:** `process { executor = 'slurm' }` (Good for blocks).
- **Variables:**
 - Use ``${projectDir}`` for paths inside your repo.
 - Use ``${launchDir}`` for the current working directory.
 - Use `env('VAR_NAME')` to grab system environment variables safely.

Managing software environment



Containers & Environments

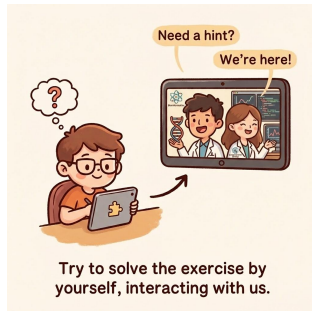
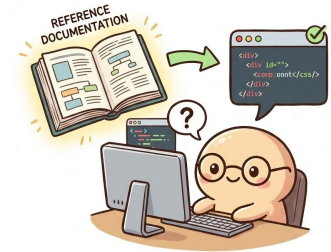
- **Technologies:** Docker, Singularity (Apptainer), Conda, etc.
- **Dedicated scopes:** Each technology is configured by a dedicated scope
- **Easy switch:** Just select your technology setting `enabled = true`.
- **Caching:** It always good to set `cacheDir` for Singularity and Conda.
 - Prevents downloading the same image 100 times.
 - On HPC clusters must be a location where you can write

Exercise - how to navigate them



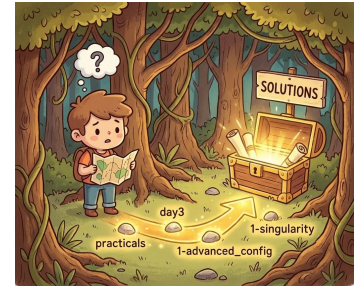
Before you start: Prepare your environment and familiarise with the example workflow as described in the exercises document > Before you start section

Reference documentation: The reference documentation file / page contains useful information and examples on each scope and parameters covered in the exercises. Use this as a support to when working on your solution.



Try to **solve** the exercise by yourself, interacting with us.

If you feel lost, **solutions** are provided for you



Exercise 1 - activate singularity

Exercise: *exercises document > Exercise 1 - Activate singularity*

Relevant documentation sections: *the basic principles, container & environment management scopes*

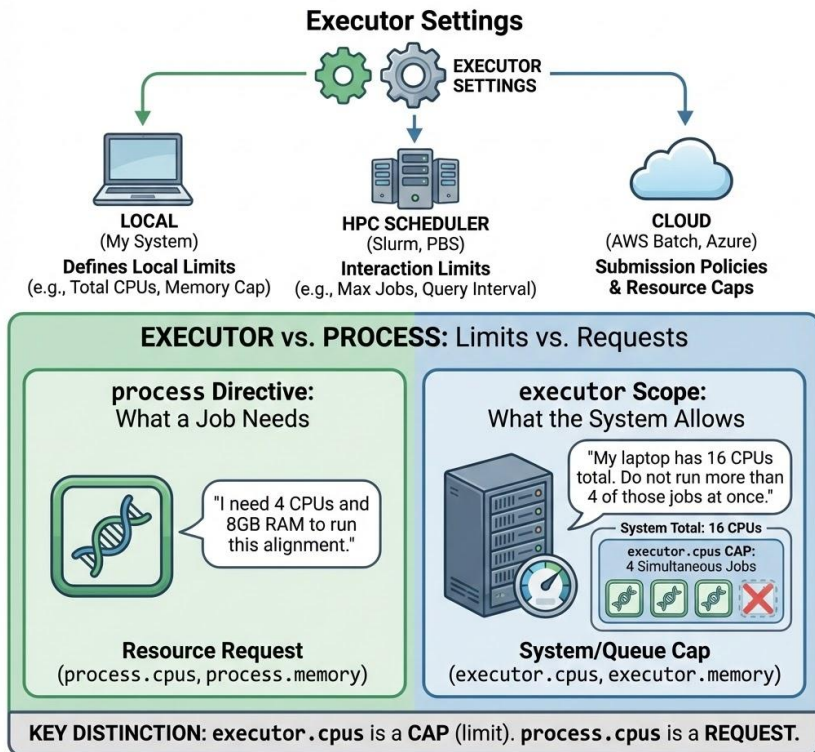
Solutions: *practicals/day3/1-advanced_config/1-singularity*



Inspect the resolved configuration: `nextflow config my/pipeline/main.nf`

The executor: Managing the Manager

NEXTFLOW EXECUTOR SCOPE: Managing the Manager



- Executor settings decide **how to actually execute your processes**: locally, submit to HPC scheduler, submit to cloud?
- Defines the **limits of the local system resources**
- Configure **interaction with the scheduling service**: maximum number of jobs, query interval for status check, etc.

Executor vs. Process

- **process Directive**: Defines what an **individual job** needs and which executor to use.
- **executor Scope**: Defines the limits of the **system/queue**.
- **Key Distinction**: **executor.cpus** is a **cap** (limit). **process.cpus** is a **request**.

Exercise 2 - set HPC executor

Exercise: *exercises document > Exercise 2 - Configure an executor*

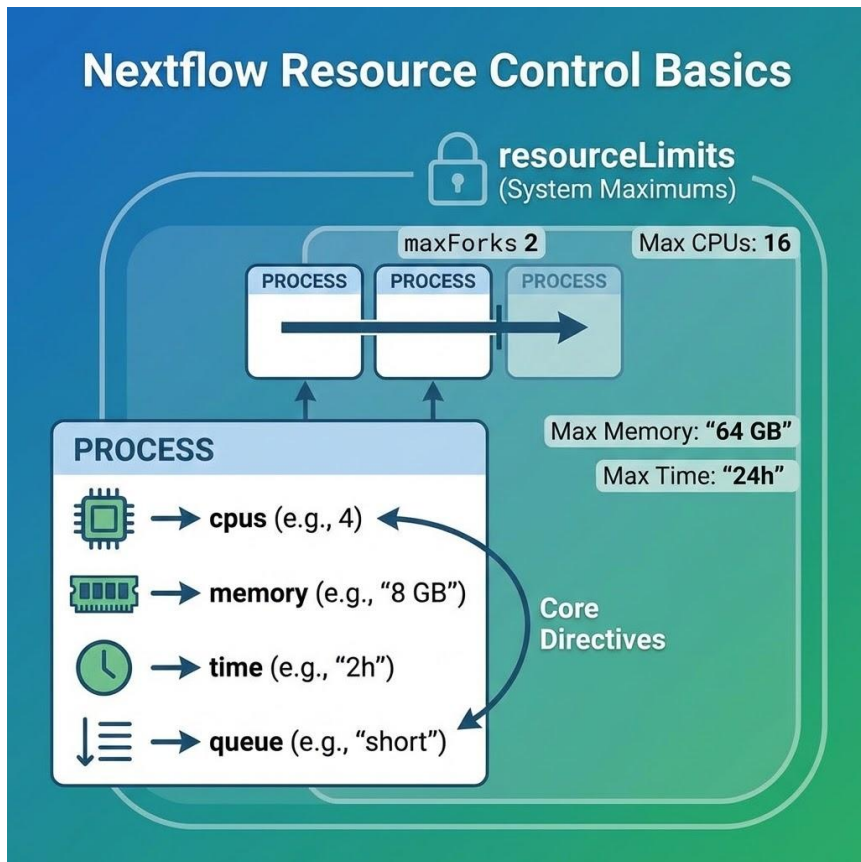
Relevant documentation sections: *Executor scope and executor control*

Solutions: *practicals/day3/1-advanced_config/2-executor*



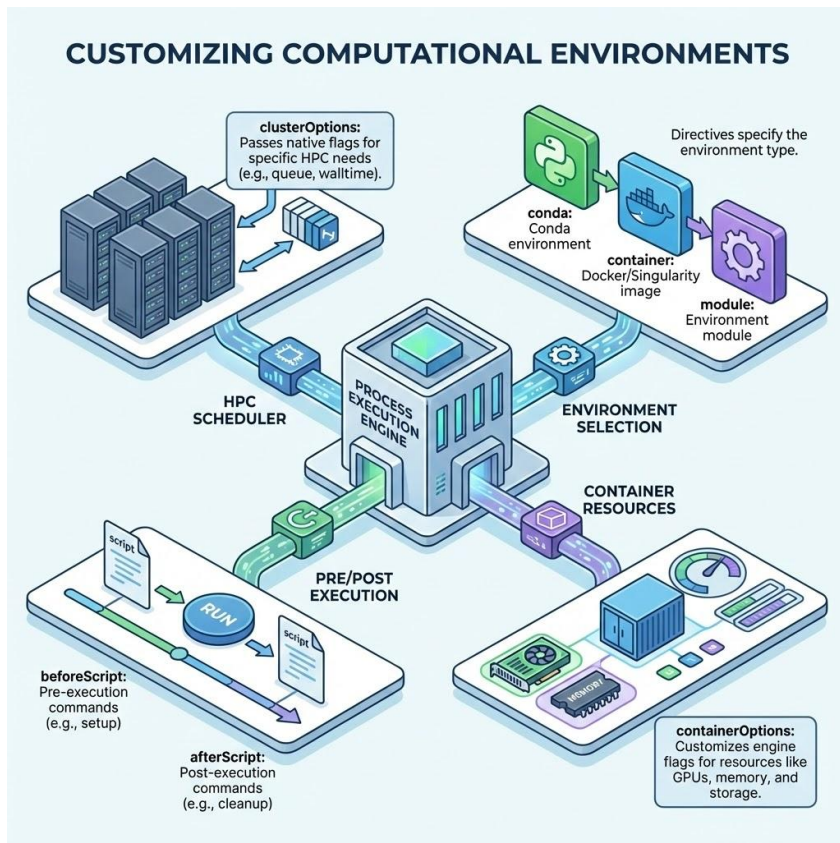
Inspect the resolved configuration: `nextflow config my/pipeline/main.nf`

Process: Resource Control



- **cpus, memory, time, queue:** Core process directives define essential resource requirements
- **maxForks:** Controls the number of parallel process instances
- **resourceLimits:** Set maximum limits for your system using the **resourceLimits** directive in the process configuration scope

Process: control the environment



Computational environment can be tailored for you process

- **clusterOptions** passes native scheduler flags for specific HPC needs.
- **beforeScript** and **afterScript** manage pre/post-execution environment commands.
- Different computational environments can be set using **conda**, **container** or **module** directives
- **containerOptions** customizes container engine flags for resources like GPUs.

Exercise 3 - Control process resources

Exercise: *exercises document > Exercise 3 - Basic process configuration*

Relevant documentation sections: *Main directives to control resource usage, Directives to control the computational environment*

Solutions: *practicals/day3/1-advanced_config/3-process_config1*



Inspect the resolved configuration: `nextflow config my/pipeline/main.nf`

Exercise 4 - Control process resources 2

Exercise: *exercises document > Exercise 4 - More on process configuration*

Relevant documentation sections: *Main directives to control resource usage, Container & Environment Directives*

Solutions: *practicals/day3/1-advanced_config/4-process_config2*

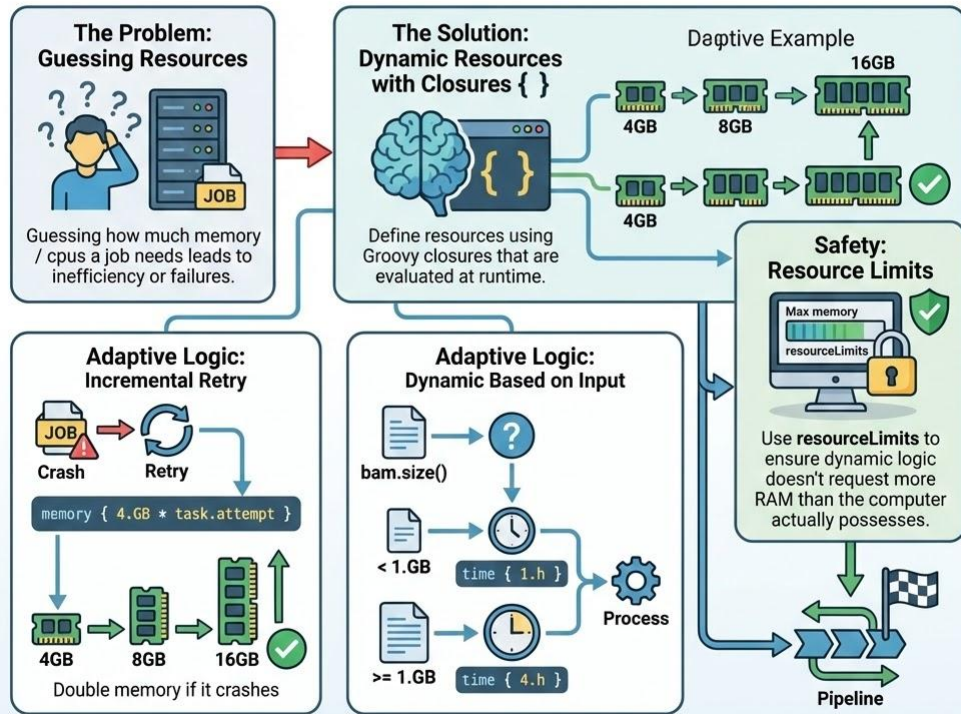


Inspect the resolved configuration: `nextflow config my/pipeline/main.nf`

Process: Dynamic resources

The 'Smart' Pipeline

Nextflow's Superpower: Automatically adapt resources to input or retry on failure



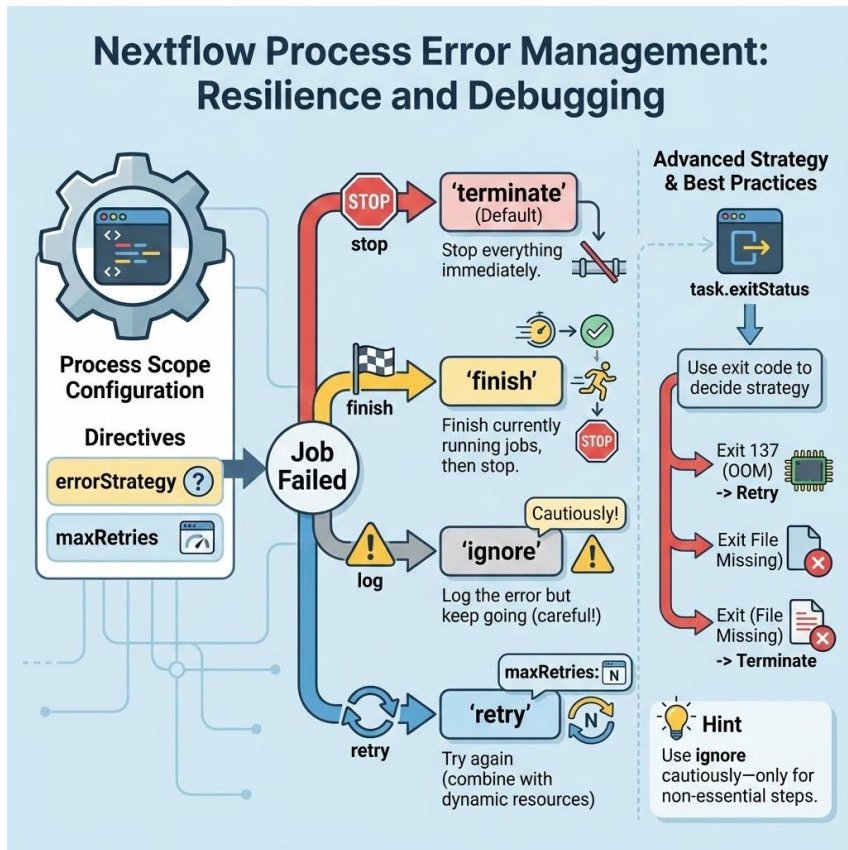
Dynamic resources using closures { }.

Adaptive Logic:

- Incremental retry
`memory { 4.GB * task.attempt }`
(Double memory if it crashes).
- Dynamic based on input properties
`time { bam.size() < 1.GB ? 1.h : 4.h }`.

Use `resourceLimits` at the process scope root to ensure dynamic logic doesn't request more RAM than the computer actually possesses.

Process: Error strategies



Process directives that controls resilience:

- **errorStrategy:** What to do when a job fails?
- **maxRetries:** How many times to try before giving up.

task.exitStatus

Capture the exit code to decide strategy (e.g., Retry on OOM 137, Terminate on File Missing).

Exercise 5 - Dynamic resources

Exercise: *exercises document > Exercise 5 - Dynamic resource allocation*

Relevant documentation sections: *Dynamic resource allocation, Control error strategy and retry*

Solutions: *practicals/day3/1-advanced_config/5-dynamic_resources*



Inspect the resolved configuration: `nextflow config my/pipeline/main.nf`

Process: Fine-Grained Process Control

Fine-Grained Process Resource Control in Nextflow Config

The Process Scope: Directives & Defaults

```
process { ... }
```

 cpus = 4

 memory = '8.GB'

 time = '2.h'

 queue = 'general'

 container =
'ubuntu:20.04'

Defaults: Apply to all processes
unless overridden.

Selectors: Targeting Specific Processes

withName: 'FASTQC' (Specific Process)



```
withName: 'FASTQC' { ... }  
  cpus = 2  
  memory = '4.GB'  
  container =  
    'quay.io/biocontainers/  
    fastqc:0.11.9--0'  
}
```

withLabel: 'big_mem' (Process Grouping)



ALIGNMENT



ASSEMBLY

```
withLabel: 'big_mem' { ... }  
  cpus = 16  
  memory = '64.GB'  
  queue = 'highmem'  
}
```

● Best Practices & Hint

Set sane defaults for everything. Use selectors for exceptions. Use labels to group similar tasks (e.g., 'gpu_tasks', 'high_memory') and configure them together for cleaner, maintainable code.

The **process** scope

- **Root directives (defaults for everything):**
 - **cpus**, **memory**, **time**.
 - **queue** (HPC partition), **container** (Docker image).
- **Selectors (Targeting):**
 - **withName: 'FASTQC'**: Apply settings only to the process named FASTQC.
 - **withLabel: 'big_mem'**: Apply settings to any process tagged with the label **big_mem**.

Use labels to group similar tasks (like "gpu_tasks" or "high_memory") and configure them together.

Exercise 6 - Fine-grained process settings

Exercise: *exercises document > Exercise 6 - Fine grained process settings*

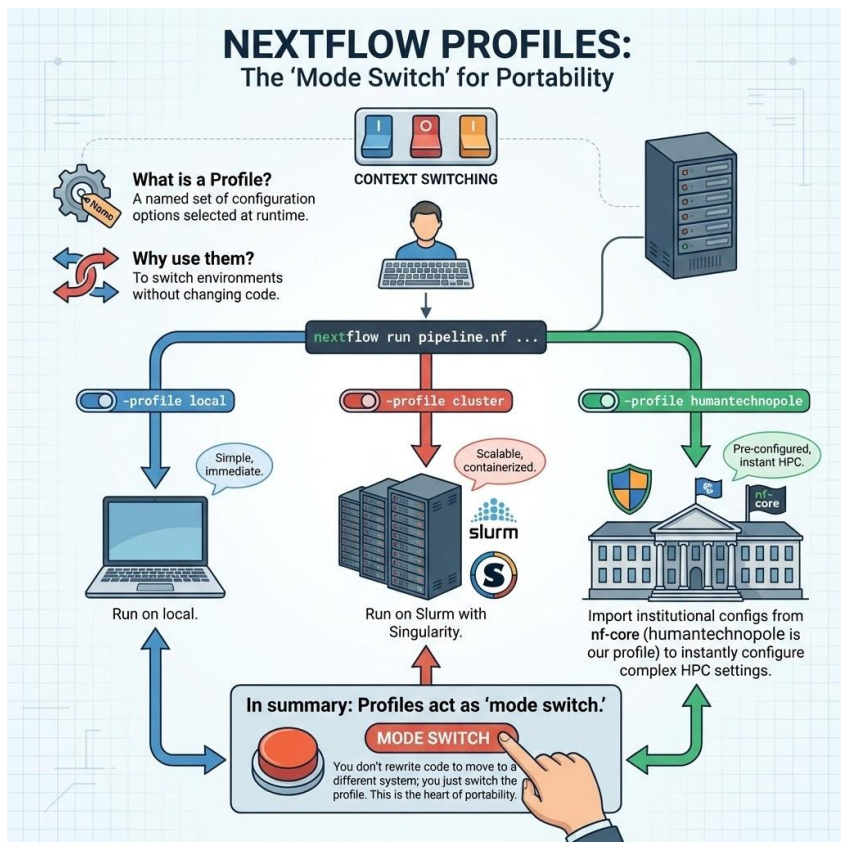
Relevant documentation sections: *Fine-grained process configuration using labels and names*

Solutions: *practicals/day3/1-advanced_config/6-fine_grained_process_config*



Inspect the resolved configuration: `nextflow config my/pipeline/main.nf`

The Power of Profiles



The **profiles** scope

- **What is a Profile?** A named set of configuration options selected at runtime. Any valid configuration scope can be set inside a profile
- **Why use them?** To switch environments without changing code.
- **Institutional Profiles:** You can import institutional configs from **nf-core** (*humantechpole* is our profile) to instantly configure complex HPC settings.

Switch the profile at command line using **-profile** to change execution environment.

Exercise 7 - Profiles

Exercise: *exercises document > Exercise 7 - Profiles*

Relevant documentation sections: *Profiles*

Solutions: *practicals/day3/1-advanced_config/7-profiles*



Inspect the resolved configuration: `nextflow config my/pipeline/main.nf`

Exercise 8 - Institutional profiles

Exercise: *exercises document > Exercise 8 - Institutional profiles*

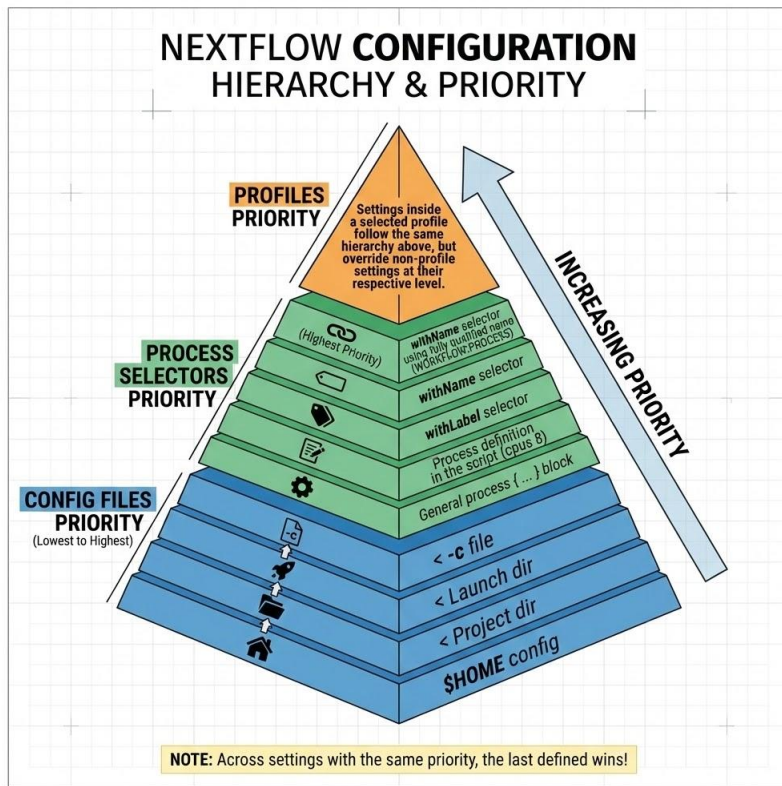
Relevant documentation sections: *Use nf-core institutional profiles, HT institutional profile*

Solutions: *practicals/day3/1-advanced_config/8-institutional_profiles*



Inspect the resolved configuration: `nextflow config my/pipeline/main.nf`

Configuration Hierarchy



Nextflow configuration priority (lowest to highest):

1. Config Files priority:

\$HOME config < Project dir < Launch dir < -c file.

2. Process Selectors priority:

- General process { ... } block.
- Process definition in the script (cpus 8).
- withLabel selector.
- withName selector
- withName selector using fully qualified name (WORKFLOW:PROCESS) (Highest Priority).

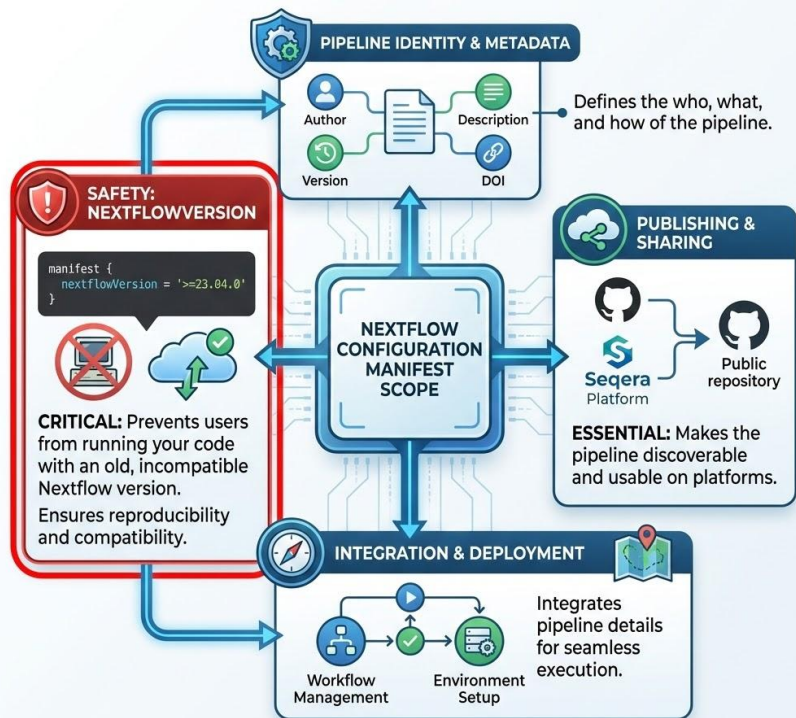
3. Profiles priority

Settings inside a selected profile follow the same hierarchy above, but override non-profile settings at their respective level.

Note: Across settings with the same priority, the last defined wins (i.e. multiple labels, multiple -c files)!

The Manifest scope

NEXTFLOW MANIFEST SCOPE SUMMARY



Pipeline Identity

- **Safety:** `nextflowVersion`

Prevent users from running your code with an old, incompatible Nextflow version

- **Metadata:** Author, Version, Description, DOI.
- **Publishing:** Essential for sharing on GitHub or Seqlera Platform.